Sorbonne Université

Master of Computer Science Computer Networks: Internet, Cybersecurity, Cloud and Automation (RES)

IOB

ChainTrack: A Blockchain-Based Technical Framework for Immutable Supply Chain Provenance

Conducted by:

ABDELWAFI Ibtissam

BOUSKRI Mohamed

Supervised by:

Prof. Maria Potop-Butucaru

Academic Year 2025/2026

Table des matières

1. Abstract 2
2. Introduction 2
2.1. The Current State of Global Supply Chains 2
2.2. The "Internet of Blockchain" (IoB) Paradigm 2
3. Problem Statement 2
3.1. Information Asymmetry 3
3.2. Counterfeiting and Fraud 3
3.3. Lack of Real-Time Visibility 3
4. Literature Review 3
4.1. Evolution of Supply Chain Management (SCM) 3
4.2. Centralized vs. Decentralized Ledgers 3
4.3. Existing Blockchain Solutions in Industry 3
5. Core Blockchain Concepts and the IOB Framework 4
5.1. Distributed Ledger Technology (DLT) 4
5.2. Smart Contracts: The Programmable Trust Layer 4
5.3. Consensus Mechanisms 4
5.4. Tokenization of Physical Assets 4
6. System Architecture 4
6.1. High-Level Overview 4
6.2. The Technical Stack (Hardhat, Ethers.js, React) 5
6.3. Data Flow and Interaction Model 5
7. Technical Implementation: The Smart Contract Layer 6
7.1. Core Data Structures 6
7.2. Functionality Deep-Dive 6
7.3. On-Chain Metadata and "Rich Details" 7
7.4.1 The Metadata Schema 7
7.4.2 Immutability of Records 7
7.4. Implementation Snippets 7
7.5. Optimization and Gas Management 8
8. Technical Implementation: Frontend Orchestration 8
8.1. Bridging Web2 and Web3 9
8.2. State Management and Reactive UI 9

8.3. User Interface (UI) Design and Functional Systems 9
8.4. MetaMask and Wallet Integration 9
9. Security Analysis and Governance 10
9.1. Access Control and Role-Based Permissions 10
9.2. Immutability and Auditability 10
9.3. Potential Attack Vectors 10
10. The IoB Perspective: Future Integrations 10
10.1. IoT and Autonomous Sensor Feedback 10
10.2. Zero-Knowledge Proofs (ZK-Proofs) 10
10.3. Cross-Chain Interoperability 11
11. Socio-Economic and Environmental Impact 11
11.1. Transparency and Ethical Sourcing 11
11.2. Reducing Administrative Overhead 11
11.3. Sustainability and Carbon Tracking 11
12. Functional Guide: How the Application Works 11
12.1. Phase 1: Product Tokenization (Creation) 11
12.2. Phase 2: Journey Logging (Updates & Transfers) 13
12.3. Phase 3: Consumer Verification (Tracking) 13
12.4. Phase 4: Lifecycle Decommissioning (Archiving) 13
12.4.1 Comprehensive Audit Trail 13
12.4.2 Authorization of Lifecycle Termination 14
12.4.3 Archival Confirmation 14
12.4.3 Final State Visualization 15
13. Case Studies and Potential Applications 15
14. Conclusion 16
References 16

Abstract

In the modern era of globalization, the journey of a single product can span across dozens of countries, multiple shipping companies, and complex regulatory environments. This complexity provides fertile ground for inefficiencies, fraud, and a significant lack of transparency.

This project presents ChainTrack, a decentralized application (dApp) designed to address these challenges through the lens of the "Internet of Blockchain" (IoB). By utilizing Ethereum-compatible smart contracts, ChainTrack provides a verifiable, immutable record of every touchpoint in a product's lifecycle.

This report details the technical architecture, implementation strategies, and theoretical foundations that make blockchain an indispensable tool for the next generation of logistics.

Introduction

The Current State of Global Supply Chains

Global trade has never been more connected, yet the systems managing this connectivity are often fragmented. Most supply chain data is stored in "walled gardens"—centralized databases owned by individual companies. This architectural choice leads to "data silos," where information is rarely shared in real-time between stakeholders. When a product is lost, damaged, or substituted with a counterfeit, the subsequent investigation is often a slow, manual process of piecing together disparate records.

The "Internet of Blockchain" (IoB) Paradigm

The "Internet of Blockchain" refers to an ecosystem where different blockchain networks and decentralized applications interact seamlessly to create a unified fabric of trust. In the context of supply chains, IoB represents more than just a ledger; it is a communication protocol where physical goods are represented as digital assets on a shared, verifiable network.

Problem Statement

Information Asymmetry

Stakeholders at the end of the supply chain (e.g., consumers) typically have the least amount of information regarding a product's origin. This asymmetry allows for "greenwashing" or ethical claims that cannot be independently verified.

Counterfeiting and Fraud

According to various industry reports, counterfeit goods cost the global economy over $500 billion annually. Current anti-counterfeiting measures (like QR codes or holograms) are often easily forged because the underlying data they point to is centralized and mutable.

Lack of Real-Time Visibility

Traditional systems are often reactive rather than proactive. Updates about a shipment location or status are usually logged after the fact, leading to delays and missed opportunities for optimization.

Literature Review

Evolution of Supply Chain Management (SCM)

SCM has evolved from simple inventory tracking to complex ERP (Enterprise Resource Planning) systems. However, ERPs are inherently internal-facing. The shift towards "Supply Chain 4.0" requires external-facing, collaborative platforms that can bridge the trust gap between disparate organizations.

Centralized vs. Decentralized Ledgers

A centralized ledger is efficient but requires a "Trusted Third Party." In many global scenarios, such a party does not exist or is too expensive to maintain across borders. Decentralized ledgers (blockchains) solve the "Byzantine Generals' Problem," allowing parties who do not trust each other to agree on a common state of data without a central mediator.

Existing Blockchain Solutions in Industry

The feasibility of blockchain in logistics is no longer theoretical. Major global initiatives have demonstrated significant returns on investment:

- **IBM Food Trust**: By linking producers, processors, and retailers (like Walmart), this consortium reduced the time to trace mangoes from the farm to the store from 7 days to 2.2 seconds.
- **TradeLens (Maersk/IBM)**: This platform digitized global shipping documents (Bills of Lading), drastically reducing the administrative "paper trail" that historically accounted for up to 20% of the cost of physical transport.
- **Provenance.org**: Focused on consumer-facing transparency, Provenance allows luxury and fashion brands to prove sustainability claims through verifiable material tracking.

ChainTrack builds on these precedents by applying the "Internet of Blockchain" (IoB) framework. While enterprise solutions are often complex and costly, ChainTrack provides a lightweight, modular alternative that demonstrates these core principles, immutability and cryptographic provenance, for small-to-medium scale operations.

Core Blockchain Concepts and the IOB Framework

Distributed Ledger Technology (DLT)

ChainTrack utilizes a Peer-to-Peer (P2P) network where every node maintains a copy of the ledger. This redundancy ensures high availability and prevents a single point of failure.

Smart Contracts: The Programmable Trust Layer

Smart contracts are self-executing contracts with the terms of the agreement between buyer and seller being directly written into lines of code. In ChainTrack, the smart contract acts as an automated auditor that only allows valid state transitions.

Consensus Mechanisms

While our implementation runs on a local Hardhat network for development, in a live IoB environment, consensus mechanisms like Proof of Stake (PoS) ensure that transactions are validated without the massive energy consumption seen in older Proof of Work (PoW) systems.

Tokenization of Physical Assets

Tokenization is the process of representing a physical good as a digital token on the blockchain. By linking a unique digital ID to a physical product, we create a "digital twin" that tracks the item's state and ownership in real-time.

System Architecture

High-Level Overview

The system architecture follows a three-tier model:

1. **Blockchain Layer**: The "Source of Truth" where the SupplyChain smart contract resides.
2. **Interaction Layer**: The middleware consisting of Ethers.js that facilitates communication between the browser and the blockchain.
3. **Application Layer**: The React-based frontend provides the user experience.

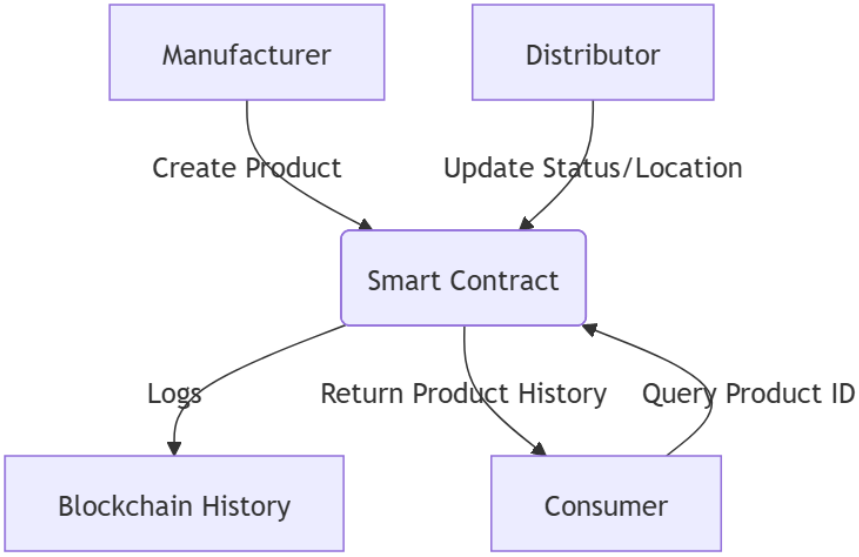
The Technical Stack (Hardhat, Ethers.js, React)

- **Hardhat**: A development environment to compile, deploy, and test Ethereum software. It provides a local EVM (Ethereum Virtual Machine).
- **Solidity**: The high-level language used for writing smart contracts.
- **Vite + React**: A fast, modern development server and UI library for building reactive interfaces.

Data Flow and Interaction Model

The interaction model is designed to maintain a synchronized state across the decentralized network while providing a responsive user experience.

1. **State Initiation**: The frontend (React) captures user input and signs it using the private keys held in MetaMask.
2. **Transaction Execution**: Ethers.js broadcasts the signed transaction to the local Hardhat node (or public Ethereum node).
3. **On-Chain Mutation**: The SupplyChain smart contract validates the transaction, updates the internal mapping, and appends a new History node to the product's array.
4. **Asynchronous Feedback**: Once the block is mined, the contract emits an event (e.g., ProductCreated). The frontend listens for these events to provide immediate confirmation to the user.
5. **Query Path**: For product tracking, the frontend makes a read-only call to getProductHistory. Since this does not mutate state, it is executed locally by the node without a gas cost, ensuring high-speed data retrieval for consumers.



Technical Implementation: The Smart Contract Layer

Core Data Structures

The contract is centered around two primary data structures: Product and History.

The use of a struct for History allows us to capture rich metadata for every transaction.

```
struct History {
   string action;
   string details;
   address actor;
```

```
   uint256 timestamp;
}
struct Product {
   uint256 id;
   string name;
   string description;
   string origin;
   address currentOwner;
   bool isActive;
   History[] history;
}
```

Functionality Deep-Dive

- **createProduct**: Initializes a new entry. The ID is auto-incremented to ensure uniqueness.
- **transferProduct**: Changes the currentOwner address. Crucially, it requires msg.sender == product.currentOwner, enforcing that only the rightful holder can pass the asset.
- **updateProduct**: Allows the current owner to append logs without changing ownership, useful for transit updates or quality checks.
- **deactivateProduct**: An administrative/ownership function that marks a product as isActive = false. This represents the end of the product's operational lifecycle (e.g., consumption, destruction, or decommissioning).

On-Chain Metadata and "Rich Details"

A core innovation of ChainTrack is its handling of metadata. Unlike traditional databases that might overwrite status fields, ChainTrack utilizes a "cumulative metadata" model.

7.4.1 The Metadata Schema

The system captures three types of metadata:

- **Identity Metadata**: Immutable attributes defined at creation (name, origin).
- **State Metadata**: Mutable fields reflecting the status (currentOwner, isActive).
- **Auditable History Metadata**: Every state change appends a History object. This includes the actor (address), timestamp (Unix time), and the details field, a flexible string allowing stakeholders to input specific context such as batch numbers, sensor readings, or logistics notes.

7.4.2 Immutability of Records

Because each metadata update is pushed to an array rather than replaced, auditors can "replay" the entire journey of a product, ensuring no stakeholder has tampered earlier logs.

Implementation Snippets

Below are key segments of the smart contract implementation that illustrate the architectural choices mentioned above.

Product Creation with History Tracking

```
function createProduct(string memory _name, string memory _description, string
memory _origin) public returns (uint256) {
   _productIds++;
   uint256 newId = _productIds;
   Product storage newProduct = products[newId];
   newProduct.id = newId;
   newProduct.name = _name;
   newProduct.description = _description;
   newProduct.origin = _origin;
   newProduct.currentOwner = msg.sender;
   newProduct.isActive = true;
   newProduct.history.push(History({
       action: "Created",
       details: string(abi.encodePacked("Origin: ", _origin)),
       actor: msg.sender,
       timestamp: block.timestamp
   }));
   emit ProductCreated(newId, _name, msg.sender);
   return newId;
}
```

Secure Ownership Transfer

```
function transferProduct(uint256 _productId, address _newOwner, string memory
_details) public {
   Product storage product = products[_productId];
   require(product.isActive, "Product is not active");
   require(msg.sender == product.currentOwner, "Only owner can transfer");
   require(_newOwner != address(0), "Invalid new owner");
   address previousOwner = product.currentOwner;
   product.currentOwner = _newOwner;
   product.history.push(History({
       action: "Transferred",
       details: _details,
       actor: msg.sender,
       timestamp: block.timestamp
   }));
   emit ProductTransferred(_productId, previousOwner, _newOwner, _details);
}
```

Lifecycle Deactivation

```
function deactivateProduct(uint256 _productId) public {
   Product storage product = products[_productId];
   require(product.isActive, "Product is already inactive");
   require(msg.sender == product.currentOwner, "Only owner can deactivate");
   product.isActive = false;
   product.history.push(History({
       action: "Deactivated",
       details: "Product marked as inactive/archived",
       actor: msg.sender,
       timestamp: block.timestamp
   }));
   emit ProductDeactivated(_productId, msg.sender);
}
```

Optimization and Gas Management

By setting the initialBaseFeePerGas to 0 in the Hardhat network configuration, the system facilitates rapid development and testing without the friction of acquiring testnet ether, while still adhering to the standard EVM transaction lifecycle.

Technical Implementation: Frontend Orchestration

Bridging Web2 and Web3

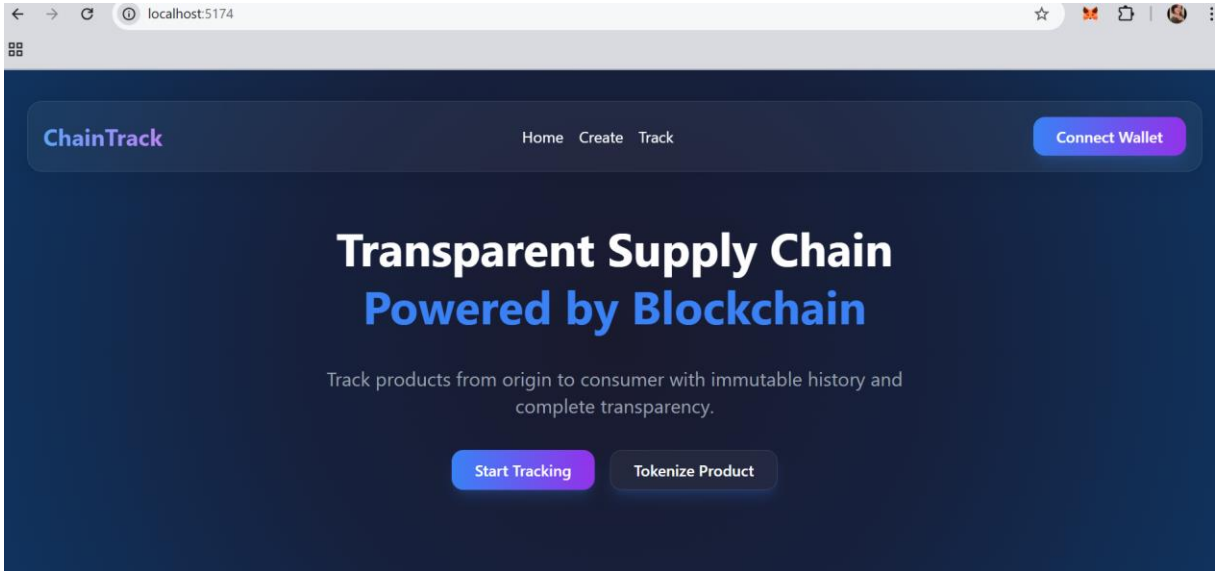
The frontend uses the ethers.BrowserProvider to tap into the user's browser-injected Ethereum provider (e.g., MetaMask). This allows for a "passwordless" experience where identity is managed by cryptographic keys.

State Management and Reactive UI

We utilize React's useState and useEffect to synchronize the UI with the blockchain state. Given that blockchain transactions are asynchronous, the UI implements intensive loading states and "optimistic" feedback to maintain a smooth UX.

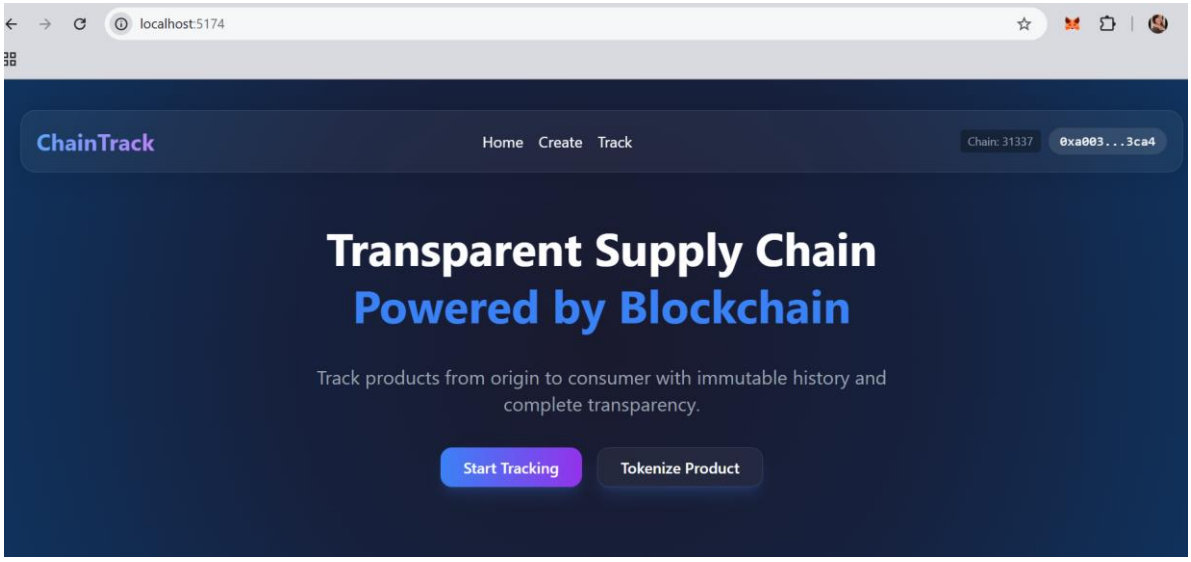
User Interface (UI) Design and Functional Systems

The interface is designed to prioritize data clarity and operational transparency. Key features include high-contrast status indicators that translate raw blockchain states (like isActive booleans) into human-readable badges. The layout is optimized for quick navigation between creation and tracking modes, ensuring that supply chain operators can manage assets with minimal cognitive load.



MetaMask and Wallet Integration

The application utilizes MetaMask as the primary gateway for user authentication and transaction authorization. This integration allows for a seamless transition from a "Guest" state to a "Connected" state, where the user's public address becomes their identity within the supply chain.



Security Analysis and Governance

Access Control and Role-Based Permissions

The implementation uses a strict ownership check. In more complex versions, this could be expanded using the OpenZeppelin AccessControl library to manage different roles like "Verifier," "Transport," and "Manufacturer."

Immutability and Auditability

Once a record is written to the blockchain, it cannot be changed. This is achieved by the fundamental structure of the blockchain, where each block contains a cryptographic hash of the previous block. Any attempt to modify history would invalidate all subsequent blocks.

Potential Attack Vectors

While the blockchain layer is secure, potential vulnerabilities exist at the interface level, such as "Phishing" or "Front-Running." Mitigations include clear transaction previews in MetaMask and robust error handling in the application layer.

The IoB Perspective: Future Integrations

IoT and Autonomous Sensor Feedback

The next frontier for ChainTrack is the automated logging of data via IoT sensors. For example, a temperature sensor in a cold-chain truck could automatically call the ***updateProduct*** function if the temperature exceeds a threshold. This removes human error and provides objective, real-time proof of handling quality.

Zero-Knowledge Proofs (ZK-Proofs)

For enterprise supply chains, privacy is often a concern. Competitors shouldn't necessarily see each other's suppliers. ZK-Proofs like *zk-SNARKs* could allow a manufacturer to prove a product is "Organic" or "Ethically Sourced" without revealing the specific supplier or farm location.

Cross-Chain Interoperability

Future versions could leverage protocols like Polkadot or Cosmos to allow the SupplyChain contract to interact with other blockchains, such as specialized logistics chains or global financial networks.

Socio-Economic and Environmental Impact

Transparency and Ethical Sourcing

By providing a verifiable path of origin, consumers can support brands that adhere to fair labor practices and environmentally friendly sourcing. This creates a market incentive for sustainability.

Reducing Administrative Overhead

By automating the verification process, companies can significantly reduce the amount of paperwork and manual auditing required, leading to leaner and more efficient operations.

Sustainability and Carbon Tracking

Blockchain can be used to track the carbon footprint of a product as it moves through the supply chain, providing a transparent record for institutional reporting and consumer awareness.

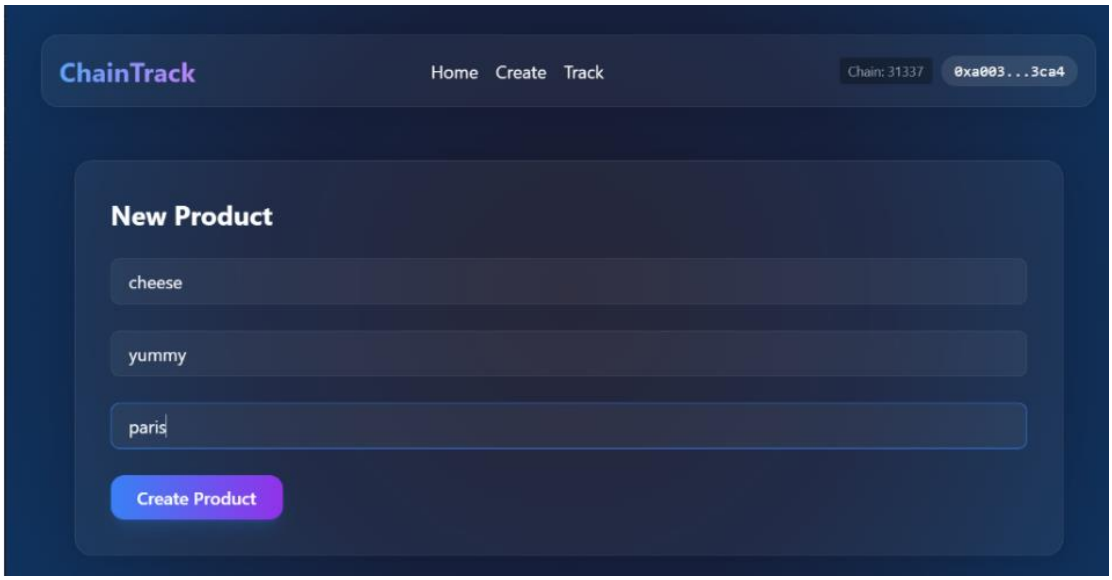
Functional Guide: How the Application Works

Phase 1: Product Tokenization (Creation)

The manufacturer initiates the record on-chain by entering the product's fundamental data (Name, Description, Origin).

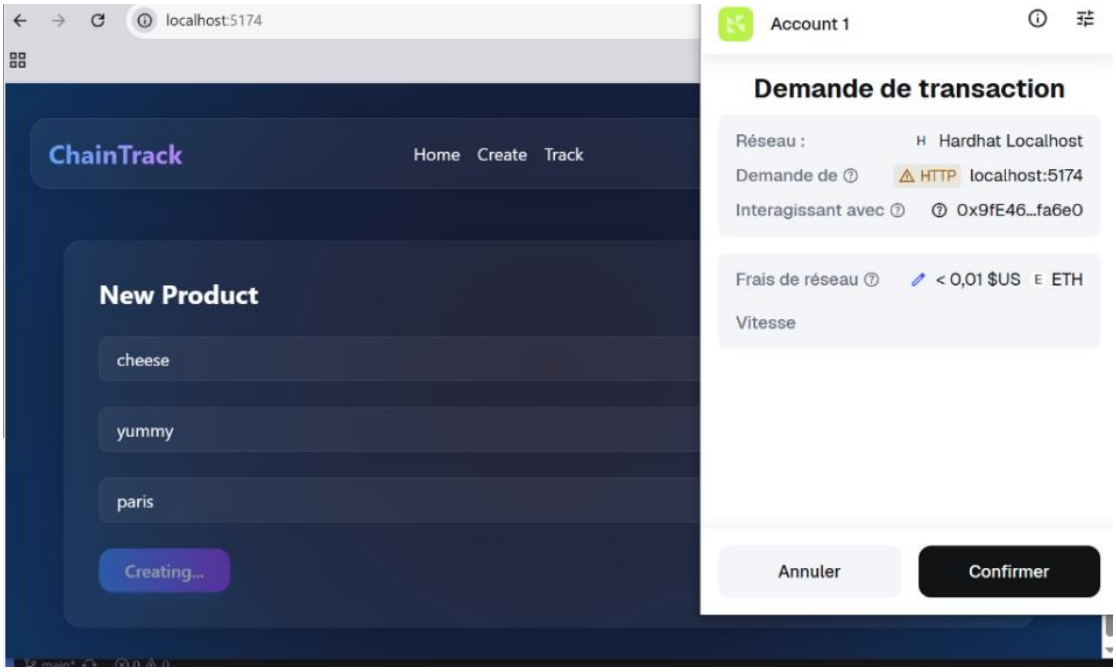
Step 1: Input Product Details

The user fills out the registration form within the "Create" interface.



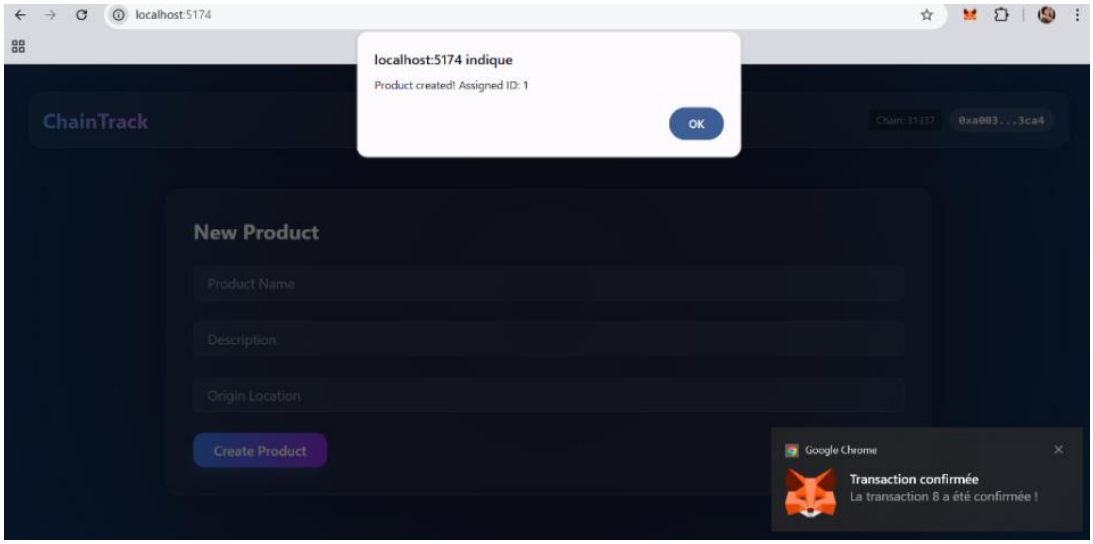
Step 2: Cryptographic Authorization

MetaMask triggers a transaction signing request, requiring the user to authorize the creation of the digital twin on-chain.



Step 3: Deployment Confirmation

A system alert confirms the successful creation and displays the unique, auto-incremented Product ID (e.g., ID 1).

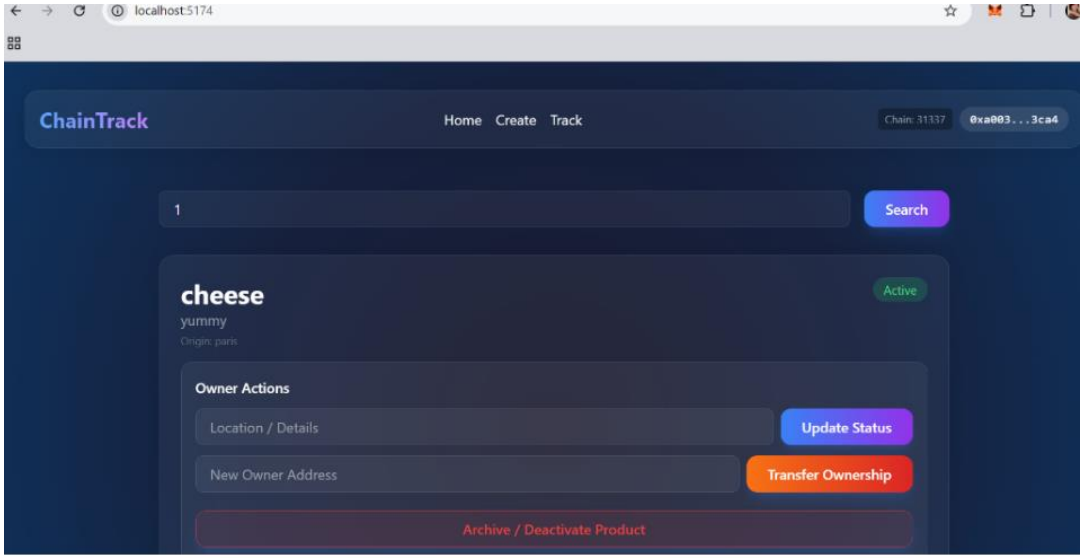


Phase 2: Journey Logging (Updates & Transfers)

As the product moves, stakeholders update its status. Each update creates a new History node. If ownership changes (e.g., to a distributor), the *transferProduct* function ensures the digital twin follows the physical asset.

Phase 3: Consumer Verification (Tracking)

Consumers enter the ID to view the "Proof of Journey."

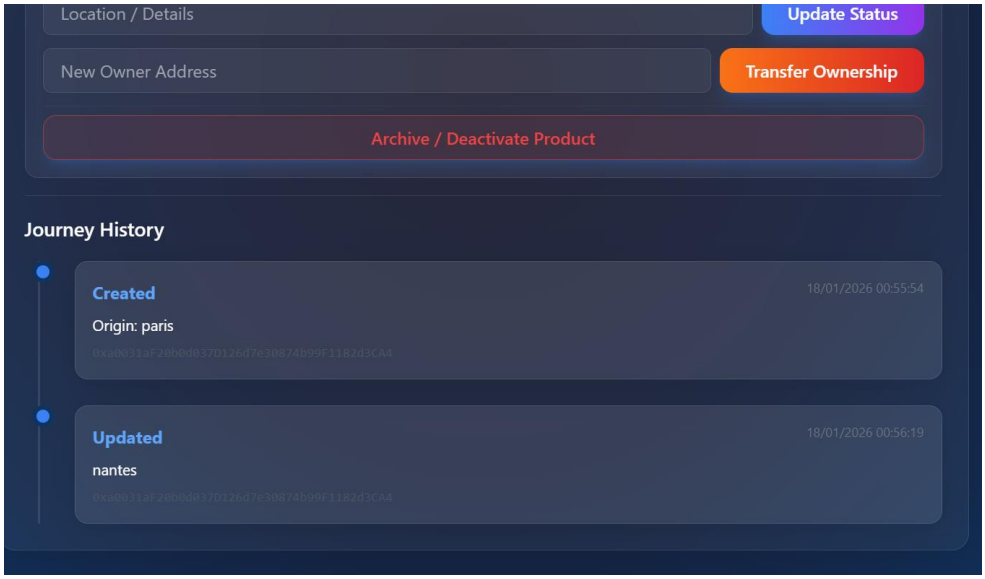


Phase 4: Lifecycle Decommissioning (Archiving)

Once a product reaches its final destination or is consumed, the owner can "Archive" it. This marks the product as isActive = false, preventing further mutations while preserving its history for future audits.

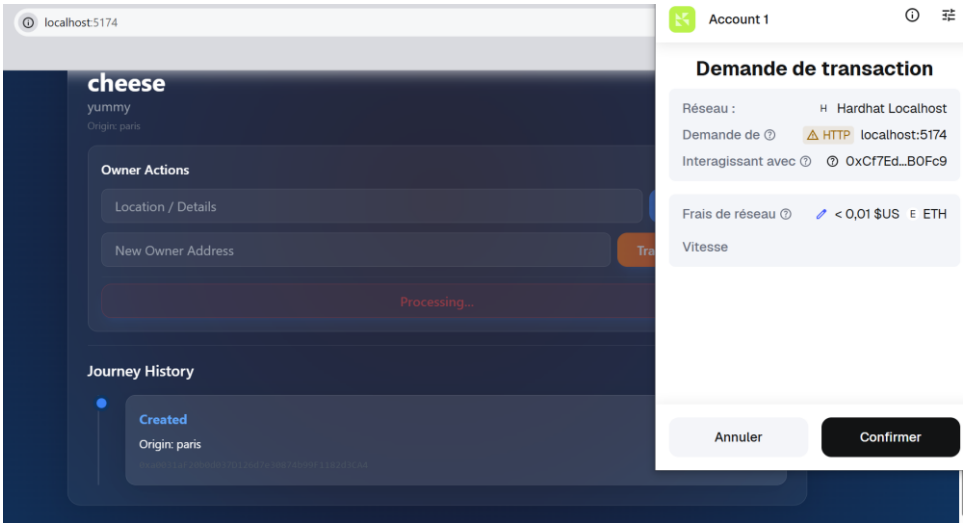
12.4.1 Comprehensive Audit Trail

The history section displays all previous actions (Created, Transferred, Updated) prior to deactivation.



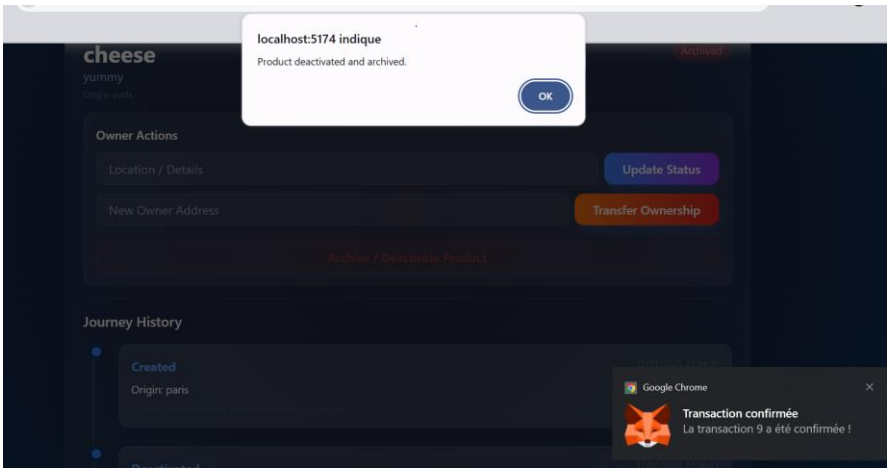
12.4.2 Authorization of Lifecycle Termination

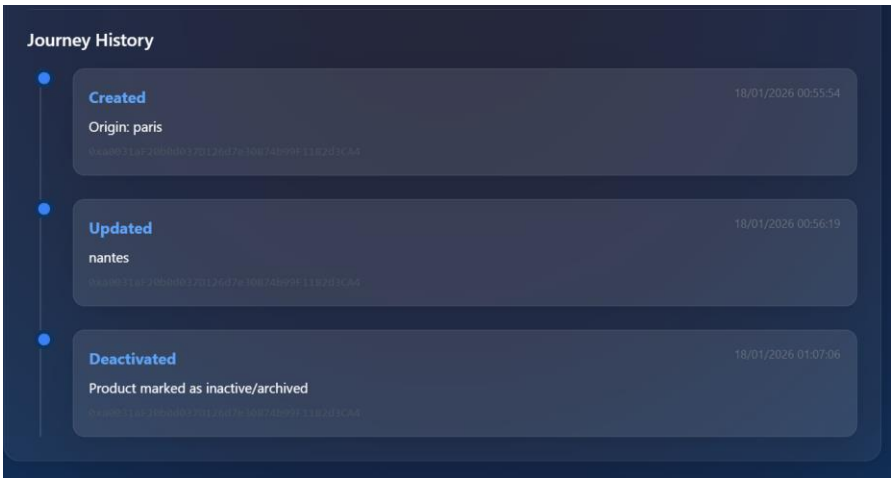
Stakeholders must sign a final transaction to archive the product, providing a cryptographic lock on its historical record.



12.4.3 Archival Confirmation

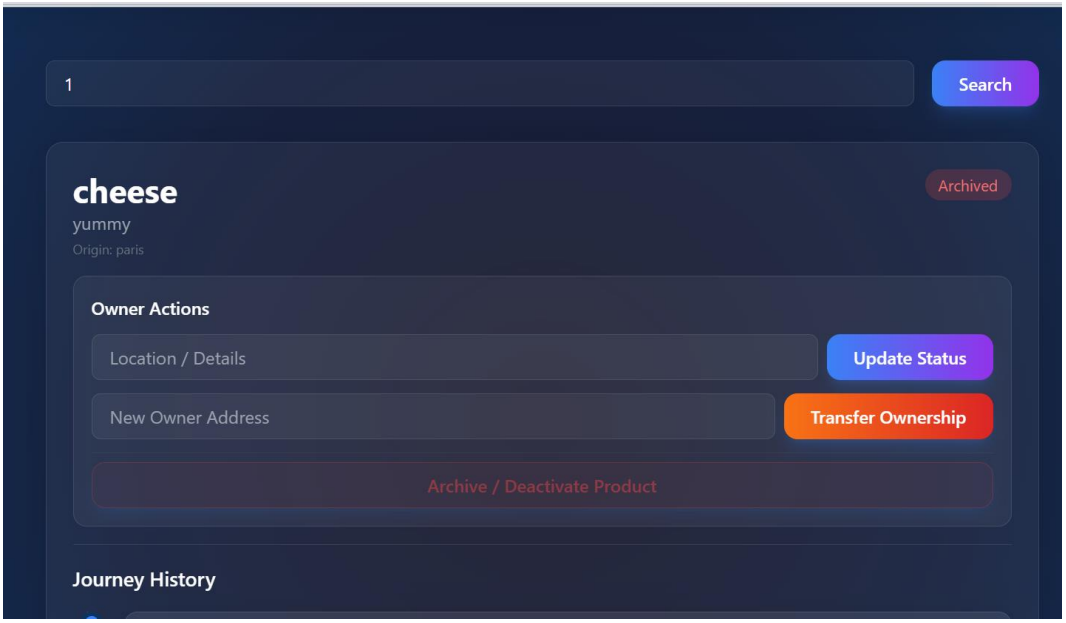
A final system notification confirms that the product has been successfully decommissioned.





12.4.3 Final State Visualization

The UI displays a clear "Archived" status badge, signaling to all viewers that the item's tracked journey is complete.



Case Studies and Potential Applications

- **Luxury Goods**: Protecting high-value items like watches or designer bags from the resale of sophisticated fakes.
- **Agriculture**: Tracking "Farm-to-Fork" for food safety, enabling targeted recalls if a specific batch is found to be contaminated.
- **Pharmaceuticals**: Ensuring that medicines are genuine and have been stored correctly during transport.

Conclusion

The ChainTrack project successfully demonstrates how the "Internet of Blockchain" can revolutionize supply chain transparency. By replacing opaque, centralized databases with a shared, immutable ledger, we move from a system based on "Trusting the Word" to one based on "Trusting the Code." While challenges such as scalability and widespread adoption remain, the technical foundation provided by EVM-compatible chains offers a robust and scalable path forward.

References

1. Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System.
2. Wood, G. (2014). Ethereum: A Secure Decentralised Generalised Transaction Ledger.
3. Tapscott, D., & Tapscott, A. (2016). Blockchain Revolution.
4. Buterin, V. (2013). Ethereum White Paper.
5. IOB Course Materials: "Foundations of Decentralized Trust."